Project, Semester 1, 2026

Submission deadline: **Friday 22 May 2026, 11:59 PM**

Total Marks: **30 (Value: 15%)**

Project description

You should construct a Python 3 program containing your solution to the given problem and submit your program electronically on Moodle. The name of the file containing your code should be your student ID e.g. 12345678.py. Remember you need to submit the program as a single file and copy-paste the same program in the provided text box. Please note that this is an individual project.

- Your program will be automatically run on Moodle for sample test cases provided in the description sheet if you click the "check" link. However, this does not test all required criteria, and your submission will be thoroughly tested for grading purposes after the due date.
- You have only one attempt to submit, so do not submit until you are satisfied with your attempt.
- All open submissions at the time of the deadline will be automatically submitted. Once your attempt is submitted, there is no way in the system to open/reverse/modify it.
- You must submit your solution before the deadline listed above. Following UWA exceptional variation of assessment (EVA) policy, a penalty of 5 per cent of the total mark allocated for the assessment item is deducted per day for the first 7 days (including weekends and public holidays) after which the assignment is not accepted. For the first two days there will be a penalty waiver, which means that if you submit within that 48-hour period, the assessment will be marked late but the late penalty will not be applied. After 48 hours, the accrued penalty will apply, i.e. a 15% deduction will be applied on Day 3 (after 48 hours), with an additional 5% per day after that (to day 7). For more information, please see the website.

You are expected to have read and understood the University's guidelines on academic conduct. In accordance with this policy, you may discuss with other students the general principles required to understand this project, but the work you submit must be the result of your own effort. Plagiarism detection, and other systems for detecting potential malpractice, will therefore be used. Besides, if what you submit is not your own work then you will have learned little and will therefore, likely, fail the final exam.

Project Overview

Similar to Lab 4, the data for this project is also the traffic data across Western Australia, but with different tasks and assumptions detailed later.

You will use the data derived from the Traffic Digest dataset from Main Roads Western Australia. The dataset shows the locations of traffic count sites on selected roads across Western Australia and provides the average number of vehicles recorded. More information can be found on the website and the web visualiser.

You are required to write a Python 3 program that will read the data in a CSV file and conduct analysis. Note that the data used in this project is different from the raw data from Traffic Digest, and you should develop your program based on the sample CSV file provided for this project. The CSV file has the following columns:

- **X**: Longitude of the traffic count site.
- **Y**: Latitude of the traffic count site.
- **SITE_NO**: Identifier for the traffic count site.
- **ROAD_NAME**: Name of the road on which the count site is located.
- **LOCATION_DESC**: Description of the site location, typically relative to a nearby road or landmark.
- **TRAFFIC_YEAR**: The financial year for which the traffic data was collected.
- **MON_SUN**: Daily traffic volume, averaged over all days of the week (Monday–Sunday).
- **MON_FRI**: Daily traffic volume, averaged over weekdays only (Monday–Friday).
- **SAT_SUN**: Daily traffic volume, averaged over weekends only (Saturday–Sunday).
- **PCT_HEAVY_MON_SUN**: Percentage of heavy vehicles in the Monday–Sunday traffic volume.
- **PCT_HEAVY_MON_FRI**: Percentage of heavy vehicles in the Monday–Friday traffic volume.
- **PCT_HEAVY_SAT_SUN**: Percentage of heavy vehicles in the Saturday–Sunday traffic volume.
- **NETWORK_PERFORMANCE_SITE**: Whether the site is a Network Performance Site ("Yes" / "No"). Network Performance Sites are permanently instrumented locations that collect traffic data, as opposed to portable equipment used for short-term sampling at other sites.
- **LG_NO**: Numeric code identifying the Local Government Area (LGA).
- **LG_NAME**: Name of the Local Government Area in which the site is located.
- **RA_NO**: Numeric code identifying the Administrative Region.
- **RA_NAME**: Name of the Administrative Region.
- **OBJECTID**: An internal object identifier for the feature record.
- **GlobalID**: A global identifier for the feature record.

To complete the task, you need to understand these data columns, which involves different terms defined by the data provider, such as “traffic count site”, “Local Government Area”, and “Administrative Region”. You are required to write a Python 3 program that will read a CSV file. After reading the file, your program is required to complete the following statistical tasks:

1) For a specific Administrative Region, summarise the data of the traffic count sites for each traffic year, including the number of sites, the average and the standard deviation of the all-day traffic volume, and the site with the highest all-day percentage of heavy vehicles.

2) For a specific Administrative Region, rank the traffic count sites within each Local Government Area, with respect to the all-day traffic volume.

3) For a specific Administrative Region, calculate the cosine similarity between the per-year averages of weekday percentages of heavy vehicles and the per-year averages of weekend percentages of heavy vehicles.

Refer to the Output section for more details.

Requirements

1) You are not allowed to import any external or internal module in python such as `csv`, `math`, `numpy`, etc.
2) Ensure your program does NOT call the `input()` function at any time. Calling the `input()` function will cause your program to hang, waiting for input that the automated testing system will not provide (in fact, what will happen is that if the marking program detects the call(s), it will not test your code at all which may result in zero grade).
3) Your program should also not call `print()` function at any time except for the case of graceful termination (if needed). If your program has encountered an error state and is exiting gracefully then your program needs to return `zero` (for number), `None` (for string), empty list (for list), or empty dictionary (for dictionary). At no point should you print the program's outputs instead of (or in addition to) returning them or provide a printout of the program's progress in calculating such outputs.
4) Do not assume that the input file names will end in `.csv`. File name suffixes such as `.csv` and `.txt` are not mandatory in systems other than Microsoft Windows. Do not enforce that within your program that the file must end with a `.csv` or any other extension (or try to add an extension onto the provided csv file argument), doing so can easily lead to syntax error and losing marks.

Input

Your program must define the function `main` with the following syntax:

```
def main(file_name, ra_name):
```

The input arguments for this function are:

- **IP1** `(file_name)`: The name of the CSV file (as string). The first row of the CSV file will contain the headings of the columns. A sample CSV file "`SampleData_Project.csv`" is provided with the description sheet on LMS and Moodle.
- **IP2** `(ra_name)`: The name of an Administrative Region **RA_NAME** (as string).

Output

We expect 3 outputs in the order below.

OP1: It will be a dictionary.

The key for each entry of the dictionary is a **TRAFFIC_YEAR** (as string) that appears in the data for the traffic count sites located within the input Administrative Region (**IP2**: `ra_name`). The value for each key is a list of four elements in the order below, calculated based on valid traffic count sites in the input Administrative Region for that traffic year:

- The first element is the number of valid traffic count sites in the input Administrative Region for that traffic year (a numeric value).
- The second element is the average of all-day traffic volume (**MON_SUN**) of these sites (a numeric value).
- The third element is the standard deviation of the all-day traffic volume (**MON_SUN**) of these sites (a numeric value).

- The fourth element is the identifier (**SITE_NO**) of the site with the highest all-day percentage of heavy vehicles (**PCT_HEAVY_MON_SUN**) among these sites (a string).

For OP1, when there are multiple sites with the same highest all-day percentage of heavy vehicles, choose the first one in alphabetical order in terms of the global identifier (**GlobalID**). If no valid traffic count site is found within the input Administrative Region, return an empty dictionary.

OP2: It will be a nested dictionary.

The keys of the outer dictionary are the names of the Local Government Areas (**LG_NAME**) that are within the input Administrative Region (**IP2**: `ra_name`) and contain at least 3 valid traffic count sites. The value for each outer key is an inner dictionary, whose keys are the identifiers (**SITE_NO** as strings) of the valid traffic count sites within that Local Government Area, and whose values are lists of three elements in the order below.

- The first element is the all-day traffic volume (**MON_SUN**) of the site (a numeric value).
- The second element is the all-day percentage of heavy vehicles (**PCT_HEAVY_MON_SUN**) of the site (a numeric value).
- The third element is the rank of the site within its Local Government Area, when the sites in that Local Government Area are sorted by the all-day traffic volume (**MON_SUN**) in descending order; the site with the highest traffic volume holds rank 1 (a numeric value).

For OP2, when there are multiple sites with the same all-day traffic volume within the same Local Government Area, rank the tied sites in alphabetical order in terms of the global identifier (**GlobalID**). If no Local Government Area within the input Administrative Region contains at least 3 valid traffic count sites, return an empty dictionary.

OP3: It will be a list of three elements in the order below, constructed from the valid traffic count sites located within the input Administrative Region (**IP2**: `ra_name`).

- The first element is a list of the averages of weekday percentages of heavy vehicles (**PCT_HEAVY_MON_FRI**) for each traffic year that appears among those sites (a list of numeric values). The list elements are in ascending chronological order according to the traffic year.
- The second element is a list of the averages of weekend percentages of heavy vehicles (**PCT_HEAVY_SAT_SUN)** for each traffic year that appears among those sites (a list of numeric values). The list elements are in ascending chronological order according to the traffic year.
- The third element is the cosine similarity (a numeric value) between the first element and the second element of OP3.

For OP3, note that the number of sites contributing to each traffic year may differ; the per-year averages must still be computed correctly. Only traffic years that have at least one valid site in the input Administrative Region are included in the first and second inner lists (otherwise this year should be skipped and not included); both lists use the same set of traffic years in the same ascending chronological order. If there are no valid traffic count sites within the input Administrative Region, return an empty list `[]` for OP3.

All returned numeric outputs must contain values rounded to **four decimal places** (if required to be rounded off). Do not round the values during calculations. Instead, round them only at the time when you save them into the final output variables.

Examples

Download "SampleData_Project.csv" file. An example of how you can call your program from the Python shell and examine the results it returns, is provided below:

```
>> OP1, OP2, OP3 = main("SampleData_Project.csv", "Metropolitan")
```

The returned output variables can be examined as follows:

```
>> OP1

{'2020/21': [20, 12039.1, 10706.8138, '53345'],

 '2021/22': [20, 22013.65, 17308.9667, '2965'],

 '2022/23': [2, 13555.0, 12119.8102, '53596'],

 '2023/24': [37, 13117.1081, 10951.7119, '53950'],

 '2024/25': [29, 18117.8621, 11750.3827, '1599'],

 '2025/26': [13, 21646.6923, 15941.0908, '4444']}

>> OP2.keys()

dict_keys(['vincent (c)', 'melville (c)', 'fremantle (c)', 'bayswater (c)',
'victoria park (t)', 'cambridge (t)', 'perth (c)', 'rockingham (c)',
'stirling (c)', 'joondalup (c)', 'canning (c)', 'cockburn (c)', 'wanneroo
(c)', 'nedlands (c)', 'swan (c)', 'armadale (c)', 'belmont (c)', 'kwinana
(c)'])

>> OP2['armadale (c)']

{'4829': [15888.0, 10.2, 1], '53596': [4985.0, 8.0, 2], '53950': [461.0,
18.9, 3]}

>> OP2['kwinana (c)']

{'8275': [33351.0, 10.5, 1], '7003': [18972.0, 15.2, 2], '8229': [6513.0,
5.5, 3]}

>> OP3

[[13.04, 8.505, 7.55, 8.3595, 7.0517, 9.0385], [7.175, 4.98, 4.85, 4.6027,
4.4793, 6.1692], 0.9963]
```

Assumptions

Your program can assume the following:

- The order of columns can be different from the order provided in the sample file. Also, there can be extra columns in the CSV file.
- Rows can be in random order except the first row containing the headings in the CSV file.

- All string data and the header in the CSV files is case-insensitive, which means "Perth" is same as "perth". Your program needs to handle the situation to consider both to be the same. Similarly, your program needs to handle the string input parameter in the same way.
- There can be missing or invalid data, for example, missing values, invalid negative values, invalid formats, duplicated rows, etc. You need to think of other cases yourself. If there is any invalid data, then entire row(s) should be ignored. For duplicated rows, all the rows should be ignored. Rows are considered duplicates if they have the same **SITE_NO**. Handling missing, invalid, or duplicated data should happen before any calculation of the outputs.
- Values can be zero and must be accounted for mathematical calculations. Your program must not crash.
- Number of Administrative Regions will vary, so do not hardcode.
- The `main` function may not be provided with valid input parameters.
- The necessary formulas are provided at the end of this document.

Important grading instruction

Note that you have not been asked to write specific functions. The task has been left to you. However, it is essential that your program defines the top-level function `main(file_name, ra_name)` (hereafter referred to as "`main()`" to save space. Note that when `main()` is written, it still implies that it is defined with its two input arguments). The idea is that within `main()`, the program calls the other functions. Of course, these functions may then call further functions. This is important because when your code is tested on Moodle, the testing program will call your `main()` function. So if you fail to define `main()`, the testing program will not be able to test your code and your submission will be graded zero. Don't forget the submission guidelines provided at the start of this document.

Marking rubric

Your program will be marked out of 30 (later scaled to be out of 15% of the final mark).

26 out of 30 marks will be awarded automatically based on how well your program completes a number of tests, reflecting normal use of the program, and how the program handles various states including, but not limited to, different numbers of rows in the input file and / or any error states. You need to think creatively what your program may face. Your submission will be graded by data files other than the provided sample data file. Therefore, you need to be creative to investigate corner or worst cases. I have provided few guidelines from ACS Accreditation manual at the end of this document which will help you to understand the expectations.

4 out of 30 marks will be awarded for style and efficiency:

• Style (2 marks): Your code should be clear and easy to read. Consider using meaningful variable names, appropriate comments, and including your name and student ID at the top of the program, etc. Python style guide

• Efficiency (2 marks): Your program should be well-structured and run efficiently.

Style Rubric:

0	Style is poor, gibberish, impossible to understand.
1	Style is fair, with small lapses, such as unclear variable names, little or no commenting, or inconsistent formatting.
2	Excellent style, really easy to read and follow.

Your program will be traversing text files of various sizes (possibly including large csv files), so you need to minimise the number of times your program looks at the same data items.

Efficiency rubric:

0	Code too complicated to judge efficiency or wrong problem tackled.
1	Very poor efficiency, additional loops, inappropriate use of `readline()`,etc.
2	Good efficiency and works well with large files, etc.

Automated testing is being used so that all submitted programs are being tested the same way. Sometimes it happens that there is one mistake in the program that means that no tests are passed. If the marker can spot the cause and fix it readily, then they are allowed to do that and your - now fixed - program will score whatever it scores from the tests, minus 4 marks per intervention, because other students will not have had the benefit of marker intervention. Still, that's way better than getting zero. On the other hand, if the bug is hard to fix, the marker needs to move on to other submissions.

Extract from Australian Computing Society Accreditation manual 2019:

As per Seoul Accord section D, a complex computing problem will normally have some or all of the following criteria:

- involves wide-ranging or conflicting technical, computing, and other issues;
- has no obvious solution, and requires conceptual thinking and innovative analysis to formulate suitable abstract models;
- a solution requires the use of in-depth computing or domain knowledge and an analytical approach that is based on well-founded principles;
- involves infrequently encountered issues;
- is outside problems encompassed by standards and standard practice for professional computing;
- involves diverse groups of stakeholders with widely varying needs;
- has significant consequences in a range of contexts;
- is a high-level problem possibly including many component parts or sub-problems;
- identification of a requirement or the cause of a problem is ill defined or unknown.

Necessary formulas

i. **Cosine Similarity:**

Mathematical formula to calculate Cosine Similarity is as follows:

$$\text{cosine similarity} = S_C(A,B) := \cos(\theta) = \frac{\mathbf{A}\cdot\mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\cdot\sqrt{\sum_{i=1}^{n} B_i^2}},$$

where $\mathbf{A}$ and $\mathbf{B}$ are two data vectors, and A_i and B_i are individual dimensions. More details can be found on https://en.wikipedia.org/wiki/Cosine_similarity

ii. **Standard deviation, *s*:**

$$s = \sqrt{\frac{\sum_{i=1}^{N}(x_i - \overline{x})^2}{N-1}}$$

where x_1, x_2, x_3 … … x_n are observed values in the sample data. $\bar{x}$ is the average value of observations and N is the number of observations.